Στόχοι:
-Δημιουργία 3 επιπέδων. (Μ)
-Εισαγωγή 3d model για κτήρια και για το σκούτερ και npc.
-Backround στις τέσσερις πλευρές.
-Διαγράμμιση ή και πεζοδόμια.
-High score logs. To score να γίνει coins.(M)
-Ο τερματισμος να γινεται με βενζίνη όχι με χρόνο. Με αντάλλαγμα score.(M)
-Η βαθμολογία να γίνεται με αρνητική βαθμολογία. Να μου δίνουν βαθμούς και να μειώνονται όσο αργούμε.
-Να επηρεάζεται κα ο αστυνόμος από συγκρούσεις με αυτοκίνητα
-Pdf με συναρτήσεις κλπ.
->Χρονόδιάγραμμα:
-Σήμερα τα επίπεδα.(Μ)(mark done)
-Αύριο ή Δευτέρα(Μ) -High score logs. To score να γίνει coins.(M)(done)
-Δευτέρα ή Τρίτη(Μ) -Ο τερματισμος να γινεται με βενζίνη όχι με χρόνο. Με αντάλλαγμα score.(M)(done tommorow adding fuel stations)
1 Μέρα:(Δ)
{-Εισαγωγή 3d model για κτήρια και για το σκούτερ και npc.
-Backround στις τέσσερις πλευρές.
-Διαγράμμιση ή και πεζοδόμια.}
1 Μέρα(Δ)
{-Η βαθμολογία να γίνεται με αρνητική βαθμολογία. Να μου δίνουν βαθμούς και να μειώνονται όσο αργούμε.
}Δ
1 Μέρα Κενό(Δ).
ΡΑΒΑΣΑΚΙ->Η δημιουργία 3 επιπέδων ολοκληρώθηκε!!!Δες στο τέλος του αρχείου για τη λογική.(Μ)
*ΡΑΒΑΣΑΚΙ2->Το κλείδωμα επιπέδων και το userlogs προστεθηκαν!Τελικά ια ήταν ωραίο να κλειδώμουν αλλά με bypass (e.g.πατώντας τη συντομευση c+s+e+v+a+s+t 😉(cheatcode)
Δες στο τέλος του αρχείου για λεπτομέρειες...
Τα επίπεδα κλειδώνουν με βάση το σκόρ του προηγούμενου(επηρεάζεται από το main γράφω αν δεις στην αρχή(93 γραμμή) του main.c).
Κανονικά θα πρέπει να γίνει με #DEFINE αλλά αυριο βράδυ!!!!
Επίσης αύριο θα δώ τα πάντα με το σκόρ και θα αξιολογήσω τις προϋποθέσεις του csevast.
Δεν είδα τις προϋποθέσεις ακόμα αλλά έβαλα καύσιμα!Τώρα μένουν τα βενζινάδικα και οι βελτιώσεις στην κίνηση που γράφω.
Τα λέμε αύριο(Μ)
SeeeYAAA!>>>

Hello και επισήμως! Επειδή σήμερα δε θα προλάβω να ασχοληθώ παραπάνω σου στέλνω feedback από αυτά που είδα από τη στιγμή που έτρεξα το παιχνίδι και στον κώδικα
1.Έχει το glitch που σου έστειλα στο ινστα, επίσης εμφανίζει σε κάποιες φάσεις ότι έχω ταχύτητα -1 ή -2 ενώ είμαι ακίνητος (μπορεί να μην παρατηρήσουν το glitch αλλά αυτό μάλλον θα το προσέξουν)
2.Έχει ένα λειτουργικό θέμα, το ότι κολλάει το αυτοκίνητο που σε καταδιώκει, αν κάτσεις στη μέση από ένα μπλοκ θα κολλήσει και θα μείνει επ αόριστον εκεί μέχρι να ξανακουνηθείς

3.Πρέπει να προσαρμόσουμε το sensitivity της κάμερας και την ταχύτητα (απλά το αναφέρω για να αμη ξεχάσουμε, ξέρω ότι το έχεις στα υπόψιν σου)
4.Δε νομίζω ότι το αναφέραμε όταν μπήκαμε στην κλήση και είναι σημαντικό, τη μουσική που πρέπει να βάλουμε(το οποίο είναι δική μου αρμοδιότητα)
5.Είδα κυριώς το header.h και το main.c και διόρθωσα κάτι τυπογραφικά στα comments, τίποτα ιδιαίτερο
6.Επίσης ο συμπιεσμένος φάκελος με όλα τα αρχεία είναι κάπου στα 2-3MB, οπότε πληρούμε και αυτήν την προϋπόθεση
7.Από αύριο αρχίζω την κυρίως δουλειά μου(διαβάζω όλο τον υπόλοιπο κώδικα, βελτιστοποιώ ό,τι μπορώ και αρχίζω τα tasks που ορίσαμε)
See you mate!!!

Λοιπόν...Μετά από 2 ώρες με ελάχιστες αυξήσεις έγινε σημαντική αναβάθμιση της ποιότητας του παιχνιδιού.

1. FullScreenMode
2. Higher FPS rate(a lot lot higher)
3. Colour imrproovement
4. Score Improovement
5. Gameover exit implemented
6. Increased smartness of the npc to reduce sticking phenomena.
   Se ya!!!Do not forget to update!!!!

Ωρα 1.40ΠΜ 10.1.2026(Μετά από Seemous)
Μολις τελείωσα τις βελτιώσεις που ήθελα για τα αυτοκίνητα στους δρόμους. Το παραδίδω στα χέρια σου.......
ΚΑλή τύχη και εύχομαι για μία αποδοτική μέρα αύριο.
Το pdf το βάζω ως word...
Έχουμε ακόμα δουλειά!!!!!!!!!!!!!!!!!!!!Αύριο θα δώ ξανά τις προϋποθέσεις.

This, is the progress file #2!
Today I tried to use differt techniques of moving a player in a world map created by me.
Steps:

1. Created a simple coordinate mooving system(if is in boundaries move)
2. Created a different always centered player system but by moving the whole world
3. Wanted to create a new intermediate system in which both the player and the system could move
4. And all of that with avoiding using the raylib camera function!It was very painfull...
5. This backup signals the end of the first step of the process. This is because we reached critical limitations.
   A. Creation of outer bounds failed due to complexity
   B. Too many arguments and variables without any need
   C. Lack of a good coordinate system
   D. Not great file and function management
   E. Even the window was not full screen
   Considering all of the limitations it is obvious that we need to start all over again.
   So what are the next steps?

1.Recreation of the same game but by using the raylib camera function(this step will likely be moved to a later time since it is complex and not mathematiclly urgent)
2.VERY IMPORTANT: Creating an algorithm with the task of finding the best possible roote between any given coordinates in the world map(see micromouse competitions)
3.Thinking and executing the main logic of the game in a small map(pickup-delivery-award-time schedule-obstacles-chasing)
4. Then full scaling
5. 3D vectoring and chasing
6.Celebrate
This is the current plan. Will see you on the next backup!
Hello Again!
This is update #4:
And we have something functional!
What we did:

6. Created a simple version with no camera movement to test the logic and the proportions of the game
7. Made a simple order pickup and dropoff system
8. Increased stability of system by implementing control before movement execution
9. Saw many youtube videos considering pseudo3d and this will hopefullly be the final touch of the game

So we have a plan and as always we set new objectives!

1. Create a new branch on git and test there all the new algorithms that will be used to test the find the best roote scenario
2. Find such algorithms on github from micromouse competitions etc
3. Add a camera system
4. Test the pseudo 3d theory and learn how it works
5. Implemengt what you learned to further improove the game
   This is the plan for the final result. Untill the next one see you!Have a nice day!

This is update #5:
What we did:

1. Implemented Time logic. Y
2. Think of a rewarding system to discuss it later. ~
3. Test differnt algoriths.
4. Based on A.I. research A* algorithm should do the job now I am trying to find ways to handle it and maybe I will test it and check some online examples. Once done we should be good and then easily draw the shortest path to the target.

Next Update!Update #6:
Today I studied the A* algorithm and quickly realized that we need to change the way we print the whole map to ensure that we can utilize and find the best possible roote.
Soo:
1.Created a function that splits the map into a grid.
2.Modified the array functions so that the map is initialized via this

3.Verified that the game still works
4.Implemented and drew the best possible roote.
THE END.-
Update #7:
Today I finally made the A* work!
Next up:
1.Add NPC'S
2.Add 2d CAMERA
3. MAKE THE GAME 3d
Play!!!!!
Seee yall
Hello again. Its me...Devastated
One day of A* work for nothing.
It is literaly trash in the NPC logic
Now I will implement standard zombie logic.
F A*. F complexity!
SAVE THIS ON THE ARCHIVE.
IT IS THE MOST TERRIBLE MOST COMPLEX CODE I HAVE WRITTEN AND IT IS TRASH>
SEEEE YALL(SORRY FOR THE RAIGEBAIT).

Updates of the last few days:

1. Buily the 3d game
2. Twuicked handling and logic
3. Added a speedometer
4. Created 3d effects.
   Next goals:
5. Put more npc's that act like walls.
6. Add the blue sky around
7. Add a better grading system and explain its logic
8. According to Mr. Katsimanis suggestion decrease the maximum speed.
9. Check if adding trafic lights is a good approach or not.
   So what we did is that we just added some cars that can move around(check out backup)
   The problem is that this move is a bit robotic and generally not nice for a quality game.
   We asked the A.I. and got these suggestions.
   Via google Gemini:
   "That system is the standard "industry trick" for background traffic in open-world games because it is computationally cheap.

To make it feel professional without overcomplicating your code, you need to add "Emergent Behavior." This means simple rules that result in complex-looking situations (like traffic jams or distinct flows).

Here are 4 logical upgrades you can implement, ordered from easiest to most impactful:

1. The "Alternating Flow" Logic (Easiest)
   Currently, all your cars likely move from Left -> Right and Top -> Bottom. This looks unnatural.

The Logic:

Use the Road Index (the i in your loop).

If the Road Index is Even (0, 2, 4...), the cars move Left to Right.

If the Road Index is Odd (1, 3, 5...), the cars move Right to Left.

The Effect: This instantly creates the illusion of two-way traffic or a busy city grid without adding any new collision logic. You just flip the speed to negative and swap the start/end points.

2. The "Brake Light" Logic (Anti-Stacking)
   Right now, if a fast car spawns behind a slow car, it will drive through it. Real cars brake.

The Logic:

Every frame, before a car moves, it checks the distance to the car directly ahead of it on the same road.

If Distance < Safe_Zone:

The car's speed becomes 0 (or matches the speed of the car ahead).

If Distance > Safe_Zone:

The car accelerates back to its normal speed.

The Effect: You will see natural traffic jams form. If one car stops (perhaps due to the traffic lights below), the cars behind it will queue up nicely.

3. The "Global Traffic Light" System
   You don't need a complex object for every intersection. You can cheat using a global timer.

The Logic:

Create a global variable: TrafficState. It cycles between HORIZONTAL_GO and VERTICAL_GO every few seconds.

Define "Intersection Zones" (ranges of X or Y coordinates where roads cross).

For Horizontal Cars:

If they are approaching an intersection AND TrafficState is VERTICAL_GO (Red Light for them):

They treat the intersection line as a "solid wall" and stop just before it.

Once the state changes to HORIZONTAL_GO, the "wall" disappears, and they accelerate.

The Effect: This creates waves of traffic. You will see empty roads, then a huge cluster of cars moving together, which adds massive realism.

4. Variable "Personalities"
   Uniformity looks robotic.

The Logic:

When you Spawn a car, assign it a random Type.

Type A (Sports Car): Small size, High speed, Bright color (Red/Yellow).

Type B (Truck): Long size (2x normal), Slow speed, Dull color (Gray/Blue).

Type C (Sedan): Average size, Average speed.

The Effect: Seeing a fast car get stuck behind a slow truck (using the Brake Light logic from #2) creates a little "story" that the player notices, making the world feel alive.

Summary of the Plan
Alternate Directions: Immediate visual variety.

Variable Speeds/Sizes: Breaks the visual pattern.

Brake Logic: Prevents overlap and allows queues.

Traffic Lights: Controls the flow and creates clumps of traffic."
So first lets try to implement the alternating pattern.
I will check all of these approaches if time is enough...We'll see. The ideas are splendid!!!

UPDATE #8:
After a very insightful talk together we laid the foundations for the next few days. So the goal for today is to create multiple levels for the game.
How this will be accomplished:

1. Create some external variables and make some defines externs(those are the parameters that will be set at the beggining of every level)
2. Create some additional parameters in the gamescreen enum
3. Create a function that sets all of those parametres at the start of every level
4. Modify the main function...
   Now lets talk about the levels...
   We need:
   -Level 1(No cars on the roads-empty city) very slow police car
   -Level 2 faster police car and some cars
   -Level 3 Full swing!Cars and crazy police guy!
   The question is...How will we be able to go from one level to another?
   Variant 1:
   -Create locked levels. How will we lock the levels? Typedef...new int which states if a level is locked...
   (Initially load these from the logs file)
   Variant 2:
   -Create settable levels from the main screen.
   Suggestion...Make the labels locked but add a cheatcode in the initial gamescreen to set all the parameters to 0.
   We will follow the suggestion.
   Lets build it!
   I have to report that we have a serious issue!And that is that all the npc logic is around a defined number!And there is nothing that can be done to change that numbers because that would result in arrays that change memory size!!!!
   So there are two options:

5. Use calloc
6. Create a big array and change the logic...
   I know that the first is the best but...I think that the second could do the same job easier(since there is npot a big ammount of data stored and we just use a partition of the arrays)
   So the second approach will be implemented to avoid increased complexity!
   Finished successfully the Variant 2.
   Instructions:
7. Press UP_ARROW from the preview screen to increase level number
8. Press DOWN_ARROW from the preview screen to decrease level number
9. Press enter to load the screen!
   Have fun!!!!
   Next up is userlogs...Lay the foundation today..Finsh tommorow!!!
   Userlogs Completed(partially).
   Next is creating locked levels!See you tommorow Jim!

Update #9:
-Todays objectives:

1. Make the levels locked(do not let the player enter an ivalid level)
2. Create a system that updates the locked levels at the completion of every game.
3. Just had a wonderfull idea!!!Next level is unlocked after player has made a ceratin highscore in the previous level!
   SO first we are going to:
4. Twick the highscore encoding in the file as -%LOCKEDLEVELNUMBER-%HIGHSCORE1-%HIGHSCORE2-%HIGHSCORE3-
   Lets do that and I will inform you about what is next!
   Tweaked the high score!
   Now it works...

Update #10: Developing a high score logic
This is the score calculation formula.

1. At the beggining the points awarded to the player are: manhantan_distance_to_pickup_from_pos + manhantan_distance_to_dropoff_from_pickup
2. We deduce points each time:
   a. We hit a car
   b. When mooving deduce every second that passes. Do not deduce if score is larger than 0.
   Did all of that!
   Tomorow I will add gas stations.
   Lets gooooooo!
   TIME LOGIC->ELIMINATED
   FUEL lOGIC->REBORN
   BACKUPED ALL.
   SEeee yALlLLL

Update #11:

1. Gas stations
   How we will implement gas station features
   a. New type gas station
   b. 3 new functions. A draw function that draws the station and a function that randomply selects a point in thne map and places a gasoline there. And an update function as well. DONE
2. Better car and npc movement with the creation of traffic lights as we have told multiple times. Not yet done
3. Full coin mode and highscore logic print when in preview. Not yet done
4. Night mode. Done

Today is npc day!!!!!